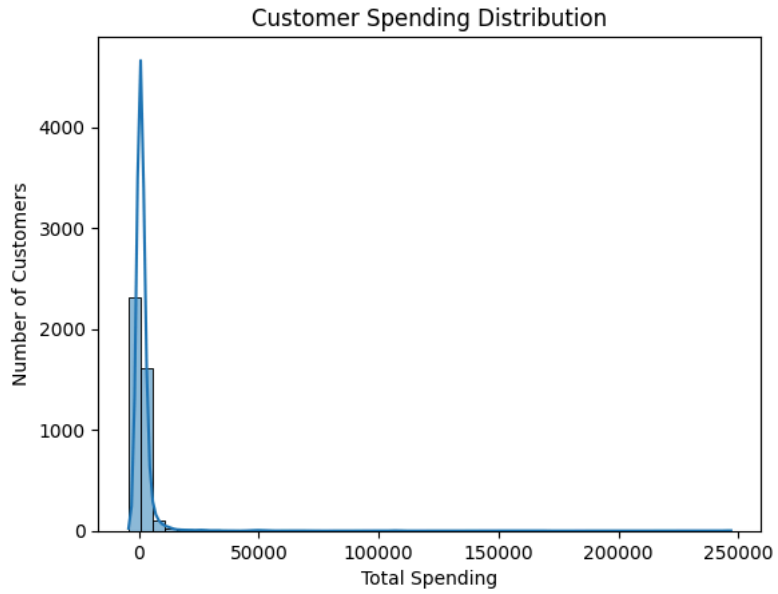
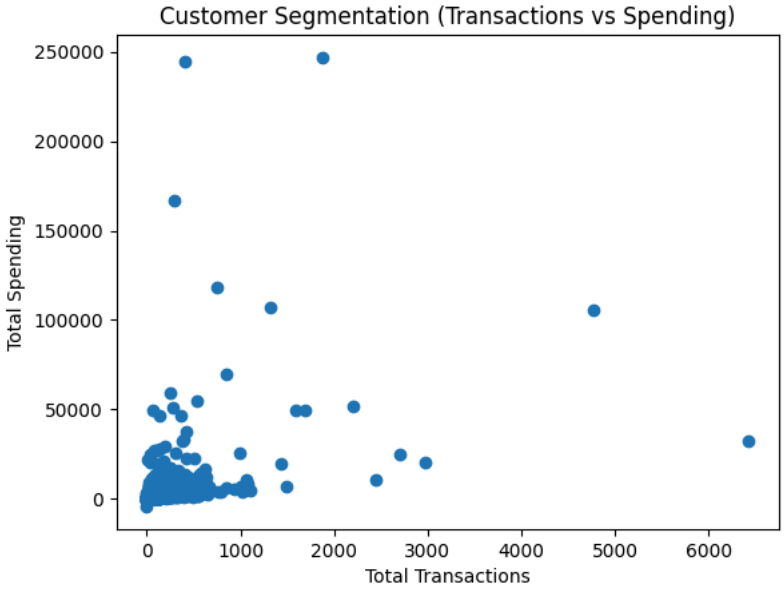
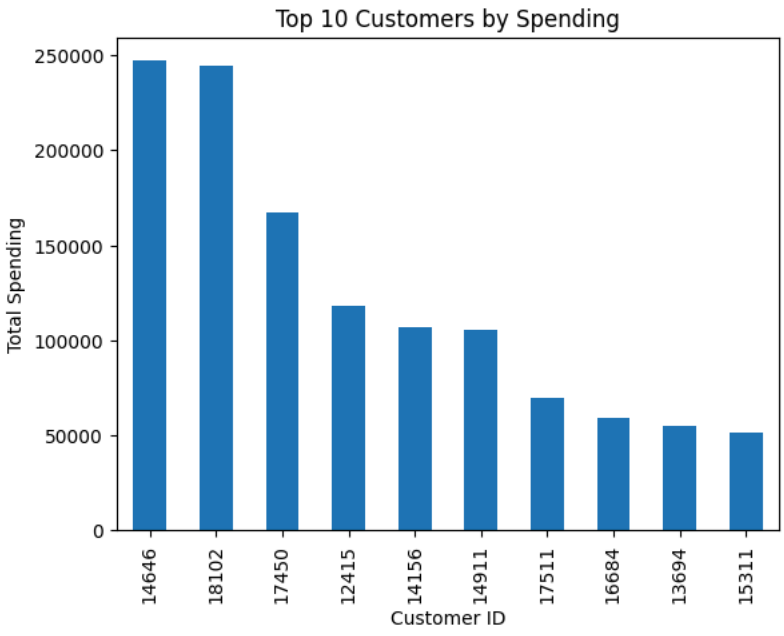
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

df = pd.read_csv('/content/OnlineRetail.csv', encoding='latin1')

df['TotalPrice'] = df['Quantity'] * df['UnitPrice']
df.dropna(subset=['CustomerID'], inplace=True)
df['CustomerID'] = df['CustomerID'].astype(int)

customer_df = df.groupby('CustomerID').agg({
    'TotalPrice': 'sum',
    'InvoiceNo': 'count',
    'Quantity': 'sum'
})

customer_df.columns = ['TotalSpent', 'TotalTransactions', 'TotalQuantity']


plt.figure()
sns.histplot(customer_df['TotalSpent'], bins=50, kde=True)
plt.title("Customer Spending Distribution")
plt.xlabel("Total Spending")
plt.ylabel("Number of Customers")
plt.show()
```



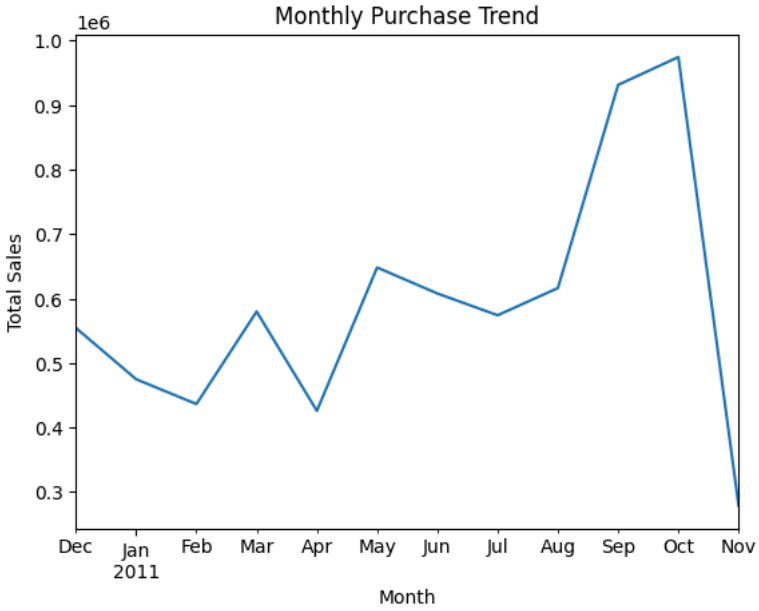

```
plt.figure()

plt.scatter(customer_df['TotalTransactions'], customer_df['TotalSpent'])

plt.title("Customer Segmentation (Transactions vs Spending)")
plt.xlabel("Total Transactions")
plt.ylabel("Total Spending")

plt.show()
```




```
top_customers = customer_df.sort_values(by='TotalSpent', ascending=False).head(10)

plt.figure()

top_customers['TotalSpent'].plot(kind='bar')

plt.title("Top 10 Customers by Spending")
plt.xlabel("Customer ID")
plt.ylabel("Total Spending")

plt.show()
```

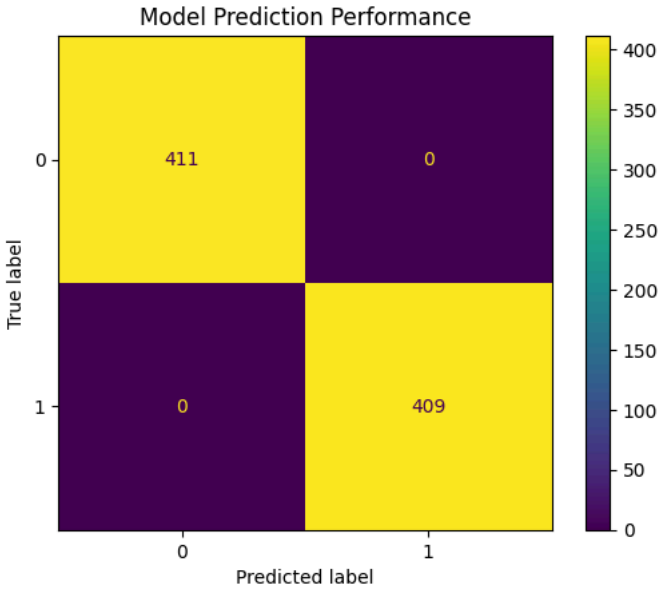



```
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
df['Month'] = df['InvoiceDate'].dt.to_period('M')

monthly_sales = df.groupby('Month')['TotalPrice'].sum()

plt.figure()

monthly_sales.plot()

plt.title("Monthly Purchase Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")

plt.show()
```



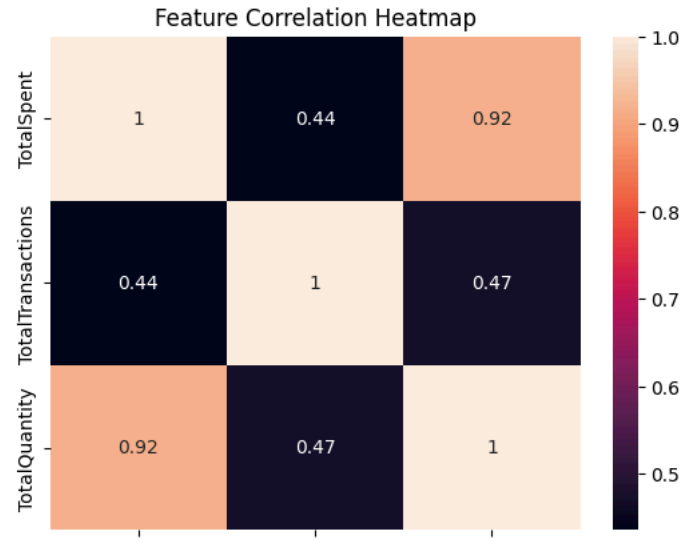

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X = customer_df[['TotalSpent', 'TotalTransactions', 'TotalQuantity']]
y = (customer_df['TotalSpent'] > customer_df['TotalSpent'].median()).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.title("Model Prediction Performance")
plt.show()
```




```
plt.figure()

sns.heatmap(customer_df.corr(), annot=True)

plt.title("Feature Correlation Heatmap")
```

```
plt.show()
```


Feature Correlation Heatmap
TotalSpent
TotalTransactions
TotalQuantity
1
0.44
0.92
0.44
1
0.47
0.92
0.47
1
1.0
0.9
0.8
0.7
0.6
0.5